# Training and Analysis of a CNN on CIFAR-10 Using Custom Dataloader

Aditya Pratap Singh (M25CSA002)

February 1, 2026

## Submission Links

- **Google Colab:** Notebook Link
- **GitHub Repository:** Source Code
- **Visualizations:** Wandb Link

**Abstract**

This report presents the design, training, and analysis of a Convolutional Neural Network (CNN) trained on the CIFAR-10 dataset. A custom data loading pipeline was implemented with an 80–20 train–validation split. The model was trained for 30 epochs, and computational complexity was analyzed using FLOPs. Gradient flow and weight update flow were visualized to study training dynamics. All experiments and visualizations were logged using Weights & Biases (W&B).

## 1 Introduction

Convolutional Neural Networks (CNNs) have achieved state-of-the-art performance in image classification tasks. CIFAR-10 is a widely used benchmark dataset for evaluating CNN architectures due to its small image size and multi-class nature. The objective of this assignment is to build a complete CNN training pipeline while emphasizing reproducibility, optimization behavior, and computational analysis.

## 2 Dataset

The CIFAR-10 dataset consists of 60,000 RGB images of size $32 \times 32$ across 10 object categories.

- Training set: 50,000 images
- Test set: 10,000 images

### 2.1 Train–Validation Split

The training set was split using an 80–20 ratio:

- Training: 40,000 images
- Validation: 10,000 images

The split was performed using index-based random permutation with a fixed random seed to ensure reproducibility.

# 3 Data Preprocessing

A custom dataset class was implemented to control transformations at the data loading stage.

## 3.1 Training Transformations

- Random horizontal flip
- Random cropping with padding
- Conversion to tensor
- Normalization with mean $(0.5, 0.5, 0.5)$ and standard deviation $(0.5, 0.5, 0.5)$

## 3.2 Validation and Test Transformations

Only deterministic transformations were applied:

- Conversion to tensor
- Normalization

This ensures that validation and test results reflect true generalization performance.

# 4 Model Architecture

A ResNet-18 architecture was selected due to its residual connections and efficient training behavior. The architecture was adapted for CIFAR-10 by:

- Replacing the initial convolution with a $3 \times 3$ kernel
- Removing the max-pooling layer
- Modifying the final fully connected layer to output 10 classes

# 5 Computational Complexity

The computational cost of the model was analyzed using Floating Point Operations (FLOPs).

Table 1: Model complexity analysis

| **Metric** | **Value** |
|---|---|
| Model | ResNet-18 |
| Input Size | $32 \times 32 \times 3$ |
| FLOPs | 557.89 MFLOPs |
| Parameters | 11.7 Million |

# 6 Training Setup

The model was trained with the following configuration:

- Optimizer: Adam
- Learning Rate: 0.001

- Loss Function: Cross-Entropy Loss
- Epochs: 30
- Batch Size: 128

A fixed random seed (42) was used to ensure deterministic behavior across experiments.

## 7 Training and Validation Performance

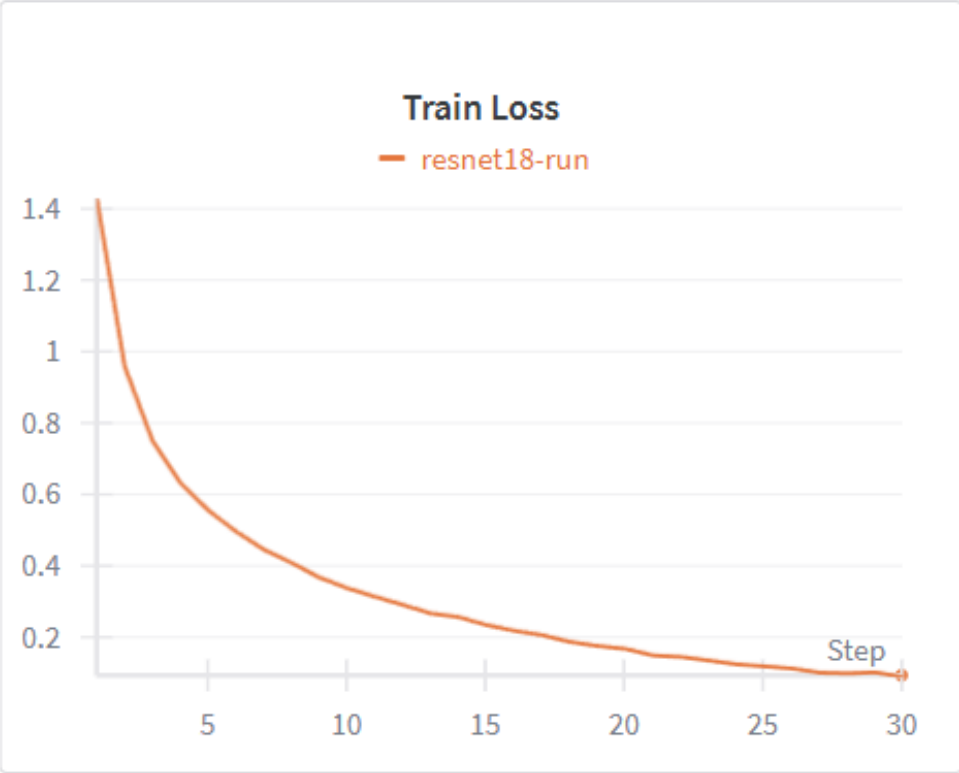


Figure 1: Training loss over 30 epochs.

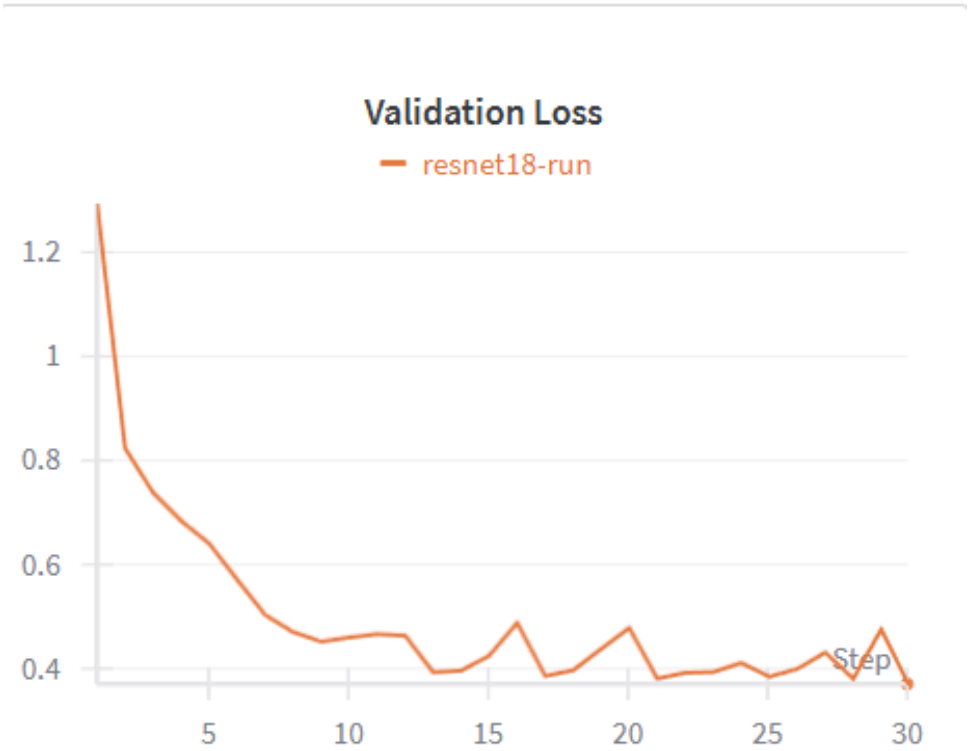


Figure 2: Validation loss over 30 epochs.

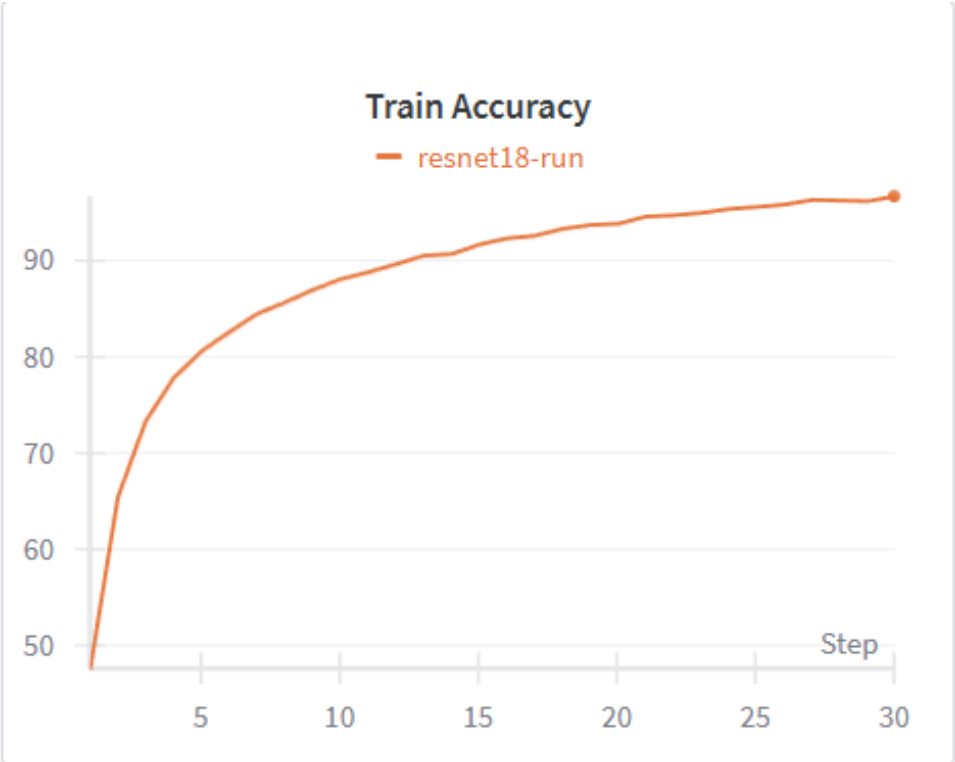


Figure 3: Training accuracy over 30 epochs.

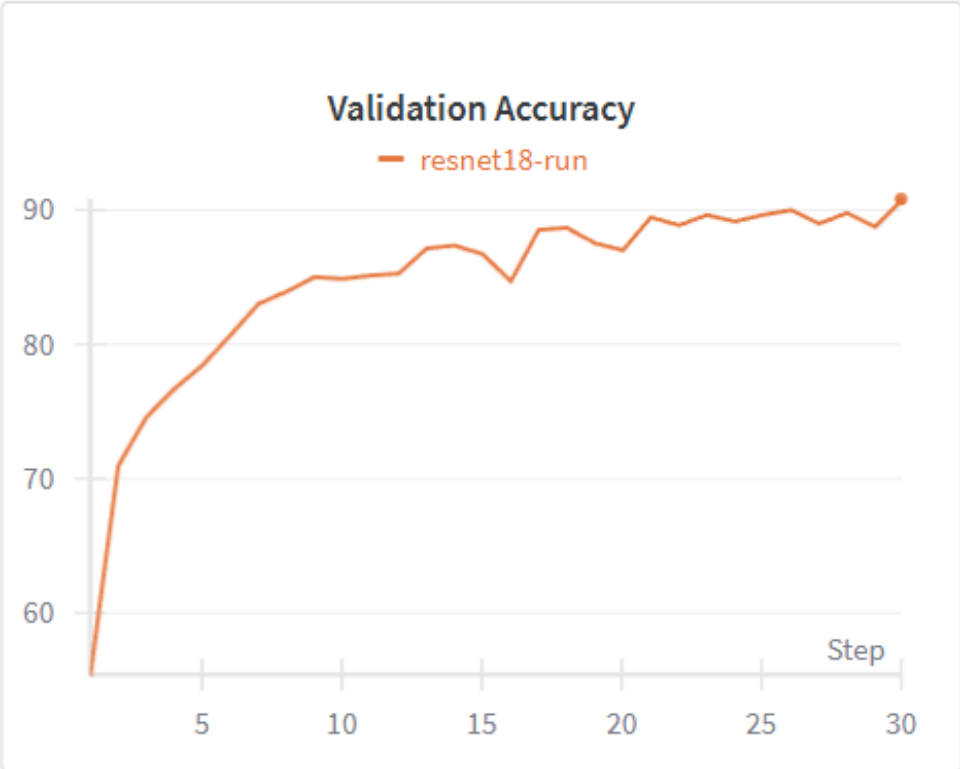


Figure 4: Validation accuracy over 30 epochs.

## 8 Gradient Flow Analysis

Gradient flow visualization was used to analyze the magnitude of gradients across network layers.

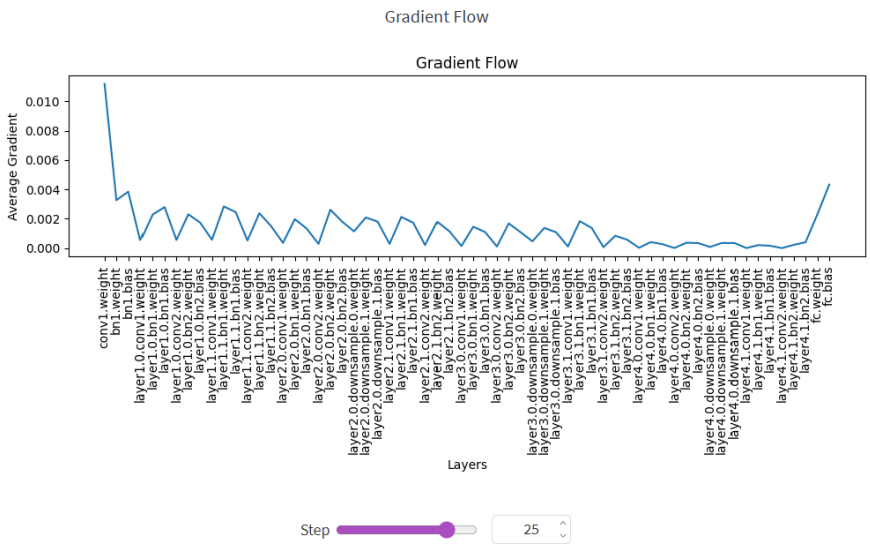


Figure 5: Gradient flow across layers (for step 25).

## 9 Weight Update Flow

Weight update flow captures the magnitude of parameter updates during training.

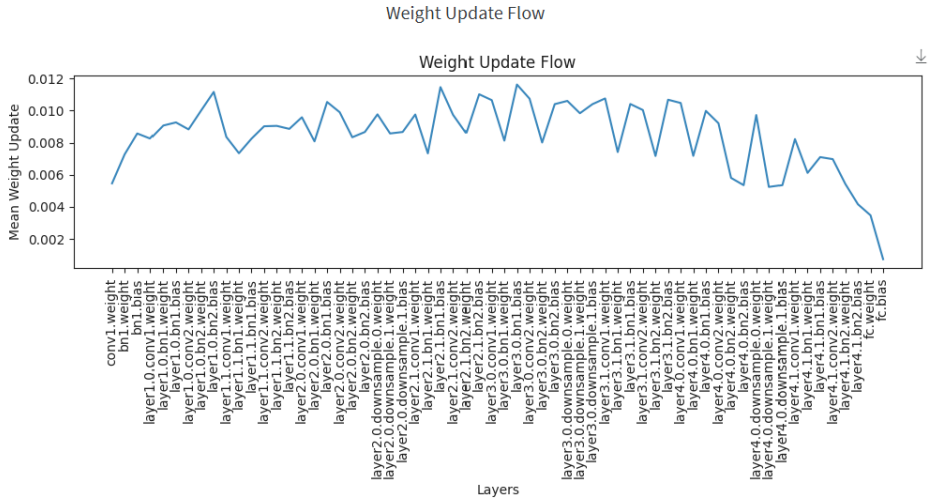


Figure 6: Weight update flow across network layers (for step 25).

## 10 Results

Table 2: Final model performance on CIFAR-10 after 30 epochs

| Dataset Split | Accuracy (%) | Loss |
|---|---|---|
| Training | 96.73 | 0.09326 |
| Validation | 90.84 | 0.37036 |
| Test | 90.35 | |

## 11 Observations

- Training and validation curves show smooth convergence with minimal overfitting.
- Gradient flow analysis confirms the absence of vanishing or exploding gradients.
- Weight update magnitudes decrease steadily, indicating effective convergence.
- ResNet-18 provides a strong trade-off between performance and computational cost.

## 12 Conclusion

This assignment demonstrated a complete deep learning workflow, from custom dataset creation to model analysis and experiment tracking. Through reproducible training, visualization of optimization dynamics, and computational complexity analysis, a robust CNN pipeline was developed. The results confirm the effectiveness of ResNet-based architectures on CIFAR-10.